CSCI 5561 Homework 0

Quenton Ni
ni000100@umn.edu

1 Introduction

In this project, I evaluate the CLIP image encoder across a variety of scenarios. I encode and cache 20 image embeddings, retrieve images from text queries using cosine similarity, and analyze both successful and failed queries. I also build an interactive Gradio interface and examine retrieval margins, prompt templates, negation, and nearest neighbor relationships.

2 Image Collection and Inspection

Twenty images were taken that spanned three major themes. These include plants (6), fruit (8), and bottled beverages (6). Among these, there are a couple of picture pairs that are meant to test CLIP. One pair was meant to test whether CLIP has a sense of how many objects there are with a picture of a single banana and a bunch of bananas. We can also test whether or not CLIP can understand backgrounds by having the same object on different backgrounds. One limitation is that the collection contains only indoor object images. This means other categories such as people, animals, and outdoor scenes cannot be evaluated. These pictures were all taken on an iPhone 17.

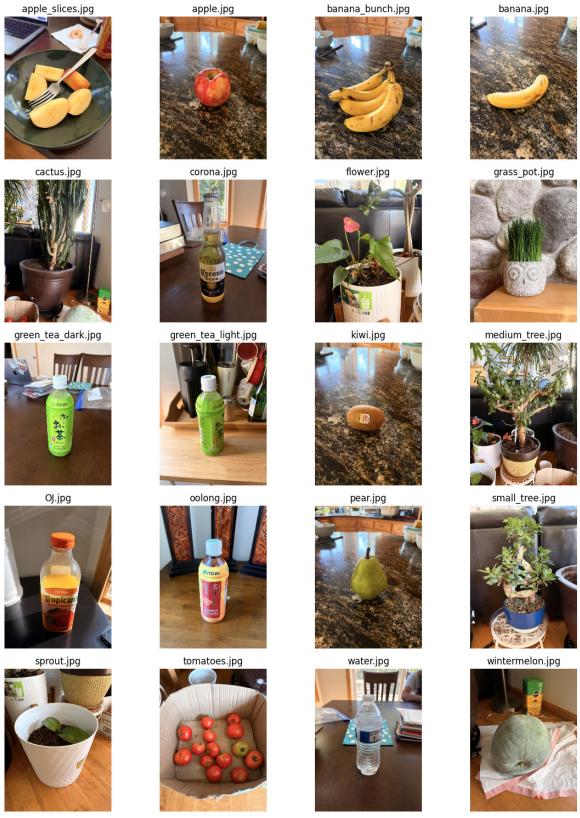


Figure 1: Grid of 20 images

3 CLIP Embeddings and Text-to-Image Retrieval

3.1 CLIP Image Embeddings

I used the OpenAI CLIP ViT-B/32 checkpoint to generate my image embeddings. CLIP preprocessing converts each image to RGB, resizes and crops it to 224 x 224 pixels, converts it to a tensor, and applies the checkpoint's normalization statistics. Each preprocessed image had shape (3, 224, 224) and type torch.float32. The complete collection of 20 images produced an embedding matrix of shape (20, 512). Before normalization, some embedding norms were 10.8539, 11.2714, and 11.2756. After

normalization, these norms were all 1. Embedding the complete collection took 0.306 seconds and these vectors were saved in memory for later use. For other runs, these vectors were also saved into a clip_embedding_cache.pt file.

3.2 Text-to-Image Retrieval

My retrieval pipeline uses CLIP to match a text query to an image within an image collection. Images are loaded with Image.open().convert("RGB"). Each image is then passed through CLIP's preprocess() and then encoded with model.encode_image(). I then normalize each image embedding and save them to a .pt file using torch.save(). In later runs I load this collection using torch.load() instead of recalculating embeddings. For each query, I use clip.tokenize() and model.encode_text() to produce a text embedding. This embedding is then normalized. Since all embeddings are unit vectors, I can compute cosine similarity against all images using a matrix vector multiplication. I can then rank the images by score using np.argsort(-scores) and display each result using Matplotlib.

4 Success and Failure Analysis

I evaluated the retrieval system with three queries. The first two queries were successful retrievals while the third revealed a failure. Each figure shows the top three images returned by the system, ordered from highest to lowest score.

4.1 Success 1: "a bunch of bananas"

This first test shows a success with the query "a bunch of bananas". In this case, there were multiple other images of fruits within the image collection. Furthermore, there was also an image of a single banana. This suggests that CLIP was able to both capture the banana category as well as a count relationship in images.



Figure 2: Retrieval results for "a bunch of bananas"

4.2 Success 2: "a photo of a cactus"

In this case, CLIP was able to differentiate the cactus from all other plant images in the collection.



Figure 3: Retrieval results for "a photo of a cactus"

4.3 Failure: "a photo of a kiwi"

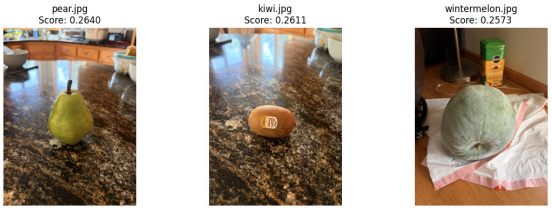


Figure 4: Retrieval results for "a photo of a kiwi"

For the query "a photo of a kiwi", I expected the image of the kiwi to rank above the other fruits. However, the pear ranked first while the kiwi image was second. This is a meaningful failure because the kiwi image is present in the collection and matches the query more directly than the highest ranked result. One possible explanation is that the sticker partially obscures the kiwi's texture, weakening some of the visual features associated with the fruit. In contrast, the pear has an unobstructed silhouette. Since the scores were very close with 0.2640 for the pear and 0.2611 for the kiwi, this obstruction could have been enough for the pear to rank first. This may suggest that CLIP's fruit representation can be sensitive to how visible an object's features are.

5 Interactive Search UI

I implemented the interactive search interface using Gradio. The interface takes a text query along with a K value to control the number of retrieved images. When a query is submitted, it is tokenized, encoded, and compared with the previously cached image embeddings. Therefore, this system does not recompute image embeddings each search. Results are displayed with rank, score, and name of image.

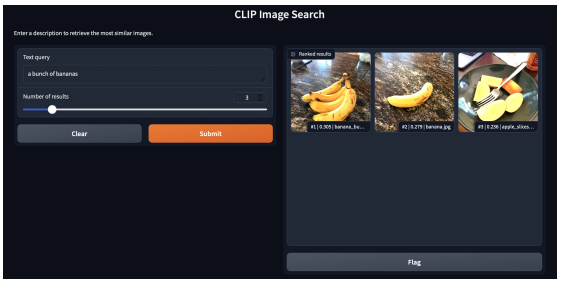


Figure 5: Gradio interface for "a bunch of bananas" with k=3

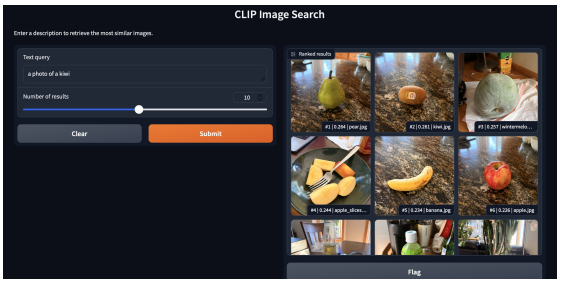


Figure 6: Gradio interface for "a photo of a kiwi" with k=10

6 Retrieval Margins

This table shows the top two similarity scores along with their difference for five different text queries. A query with a clear winner is "A bunch of bananas". The system correctly placed the bunch of bananas above the singular banana. One query that had a more ambiguous ranking was "Green tea on a light table" which had a margin of only 0.0048. While the intended image still won, the margin was small which indicates that the table color attribute may not have had a strong effect on the image representation.

Query	Top result	Score	Second result	Score	Margin
A photo of a cactus	`cactus.jpg`	0.2679	`grass_pot.jpg`	0.2431	0.0248
A bunch of bananas	`banana_bunch.jpg`	0.3049	`banana.jpg`	0.2790	0.0259
Green tea on a light table	`green_tea_light.jpg`	0.2604	`green_tea_dark.jpg`	0.2556	0.0048
A photo of a kiwi	`pear.jpg`	0.2640	`kiwi.jpg`	0.2611	0.0029
A small tree	`medium_tree.jpg`	0.2731	`small_tree.jpg`	0.2631	0.0100

Table 1: Retrieval results and margins for five text queries.

7 Prompt Templates

Prompt templates produced meaningful changes within image rankings. I used two fixed queries including "tea" and "apple slices". The templates were "a photo of [query]", "an image containing [query]", and "a close-up photo of [query]". For "tea", both the photo and close-up query change rankings compared to the raw query. The most consequential change was the close-up query as that was the one that pushed the two green tea images above the orange juice image. For "apple slices", the only and most consequential query was with the close-up template, which swapped the apple and apple slices image.

Template	Rank 1	Rank 2	Rank 3
Base query: “tea”			
Raw	Orange juice	Cactus	Sprout
Photo	Orange juice	Green tea (dark)	Sprout
Containing	Orange juice	Cactus	Sprout
Close-up	Green tea (dark)	Green tea (light)	Oolong
Base query: “apple slices”			
Raw	Apple slices	Apple	Pear
Photo	Apple slices	Apple	Pear
Containing	Apple slices	Apple	Pear
Close-up	Apple	Apple slices	Pear

Table 2: Rankings produced by the raw queries and three prompt templates.

8 Negative and Relational Language

I compared three different pairs of prompts that differed by negation. The first two queries, involving bananas and green tea, did not produce meaningfully different results. For both of them, the top two results stayed the same while the third changed. However, the negation of the tree prompt produced meaningful changes. The original query "A photo of a tree" gave the medium tree image as the top result while the negation gave completely different images for the top two rankings with the apple and banana images. Overall, CLIP’s response to negation seems rather inconsistent and only works in certain contexts.

Query	Rank 1	Rank 2	Rank 3
A photo of a banana	Banana	Banana bunch	Pear
A photo without a banana	Banana	Banana bunch	Apple slices
A photo of green tea	Green tea dark	Green tea light	Sprout
A photo without green tea	Green tea dark	Green tea light	Oolong
A photo of a tree	Medium tree	Cactus	Pear
A photo without a tree	Apple	Banana	Pear

Table 3: Rankings for positive and negated queries.

9 Nearest-Pair Comparison

I computed the cosine similarity between every pair of image embeddings to find the five most similar pairs. The two banana images and tree images had the highest similarity, due to them having the same subject category, color, and background. The apple

and pear had a similar shape, fruit category, and background. The cactus and medium tree are both indoor potted plants, while the banana bunch and pear are both fruits on the same background. These results suggest that CLIP considers object category, visual appearance, and background context when measuring image similarity.

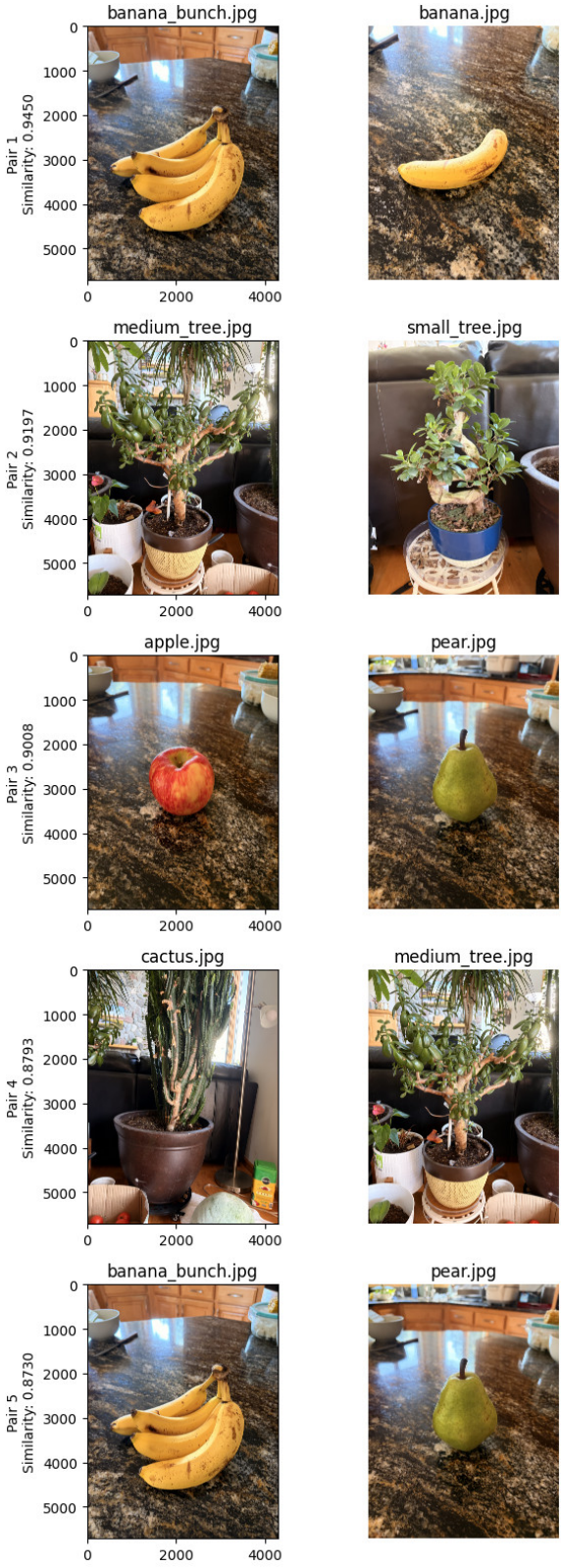


Figure 7: Five most similar image embedding pairs.